Smart Parking System — Final Technical Report
Edge Occupancy Detection with Quadrilateral Pooling, YOLOv8-cls, and a Find My Car Application

Bakhtiyor Ganijon 22013100 · Sadriddinov Otabek 21012964 · Sattor Mamatov 22012880 · Mirzayev Komronkhon 22013131

Abstract

This report presents an edge-based smart parking system that detects, for each parking space, whether it is occupied or free from a single overhead camera and reports the result as compact JSON — avoiding both the per-space cost of in-ground sensors and the bandwidth and privacy cost of streaming video to the cloud. The system is a two-stage pipeline built on the ACPDS dataset: each parking space's annotated quadrilateral *is perspective-warped into a clean* 128×128 *patch and classified as occupied or free by a 2.83 MB YOLOv8n-cls model. On the held-out unseen-lot test split the promoted model reaches* ***97.72 %*** *accuracy (0.9715 F1), matching the dataset paper's ResNet50 baseline with roughly* $9\times$ *fewer parameters, and a controlled ablation shows quadrilateral pooling beats axis-aligned square crops by 1.34 points on test. Per-spot states are temporally smoothed and served through a FastAPI backend to a web owner-setup flow, web and mobile live-occupancy maps, and a photo-based* Find My Car *feature (SIFT localization, 21/21 top-1 on the labelled query set). Stage 2 runs at 155–858 FPS across PyTorch, ONNX, and Core ML INT8 backends. The system began as a fixed-ROI pipeline that reached 90.9 % accuracy; the quadrilateral redesign documented here is what lifted it to the academic baseline. Code is available at* `https://github.com/thebkht/smart-parking-system`.

1. Introduction

Real-time information about which parking spaces are free lets drivers be routed straight to an open spot, cutting the circling that wastes time and fuel. Two hardware approaches are common: a sensor in every space, which is expensive to install and maintain, and a single camera that watches many spaces at once. The camera approach is far cheaper per space, but the naive version streams video to a server for processing — raising bandwidth cost and exposing imagery of people and license plates. This project takes the camera approach but pushes *all* inference onto the edge device, so only a small occupancy summary ever leaves it.

The system is a two-stage pipeline. Stage 1 takes each parking space's annotated quadrilateral and perspective-warps it into a clean 128×128 top-down patch; Stage 2 classifies that patch as occupied or free with a compact YOLOv8n-cls model [2]. Per-spot states are temporally smoothed and posted as JSON to a FastAPI backend:

quadrilateral polygons $\rightarrow$ *perspective warp* $\rightarrow$ *YOLOv8-cls* $\rightarrow$ *temporal smoothing* $\rightarrow$ *JSON* $\rightarrow$ *FastAPI*

Around this core, the product adds a web owner-setup flow that publishes a lot's quadrilateral layout, a live occupancy map on both web and mobile, and a photo-based *Find My Car* feature.

The key findings are: (i) on the ACPDS unseen-lot test split the promoted 2.83 MB YOLOv8n-cls reaches 97.72 % accuracy, matching the dataset paper's ResNet50 baseline at roughly $9\times$ fewer parameters; (ii) a controlled ablation confirms that quadrilateral pooling beats axis-aligned square crops, the design decision at the heart of the pipeline; and (iii) the model runs at 155–858 FPS on commodity hardware, hundreds of times faster than the system's reporting cadence requires. This system did not start in its current form — an earlier fixed-ROI design reached only 90.9 % accuracy — and that trajectory appears throughout the report as evidence that the quadrilateral redesign was the decisive change.

2. Problem statement

The objective is per-space parking occupancy detection that runs entirely on an edge device and *generalizes to parking lots not seen during training*, using only a camera — no per-space hardware — while keeping raw imagery on the device. Reaching that objective required solving two concrete problems that an earlier iteration of this project left open:

- **Localization geometry.** A parking space viewed from an angled overhead camera is a perspective quadrilateral, not an axis-aligned rectangle. Cropping a rectangle around it pulls in pixels from neighboring spaces and road markings and leaves the space perspective-distorted, which caps how cleanly a classifier can read it.
- **Classification accuracy on unseen lots.** The earlier fixed-ROI pipeline classified rectangular crops and reached only 90.9 % accuracy at its deployed threshold — well short of the ∼98 % baseline the ACPDS dataset paper [3] reports on unseen lots. Closing that gap, without growing the model past what an edge device can run, was the central technical target.

A secondary requirement was that the system be a usable product, not just a model: an owner must be able to define a lot's layout, drivers must see live occupancy on web and mobile, and a driver must be able to find their parked car from a photo.

3. Solution

Figure 1 gives the end-to-end architecture. Each parking space's annotated quadrilateral is perspective-warped to a 128×128 patch; a single compact YOLOv8n-cls model classifies every patch as occupied or free; the per-spot results are temporally smoothed, serialized to JSON, and posted to a FastAPI backend that serves the web and mobile clients. The rest of this section describes each stage in turn.

3.1. Quadrilateral pooling

The localization problem is solved by the dataset's annotation format itself: ACPDS annotates every parking spot as a four-corner *quadrilateral*, not a bounding box. These quadrilaterals provide deterministic localization for a static camera, and — unlike rectangular crops — they enable a geometrically correct patch extraction. The ACPDS paper compares the two extraction methods, reproduced in Figure 2.

The pipeline adopts method (a). Each quadrilateral is passed through `order_corners()`, which enforces a clockwise top-left → top-right → bottom-right → bottom-left ordering — without it, `getPerspectiveTransform` produces mirrored or twisted warps. The ordered corners define a homography, and `warpPerspective` maps the spot to a flat, top-down 128 × 128 patch. For new cameras without ACPDS-style annotations, the owner-setup flow (Section 6) generates the quadrilateral layout from uploaded photos; axis-aligned ROIs remain in the codebase only as a legacy fallback.

Variant	Params	Size	Role
YOLOv8n-cls	2.7 M	2.83 MB	Primary (edge)
YOLOv8s-cls	6.4 M	9.78 MB	Comparison
YOLOv8m-cls	17 M	30.22 MB	Comparison

Table 1: Stage 2 classifier variants. All use a 128 × 128 quadrilateral-pooled input and a binary occupied/free head.

3.2. Stage 2 classifier and training

Three YOLOv8-cls variants were considered, all taking the same 128 × 128 quadrilateral-warped patch as input and emitting a binary occupancy score. The nano variant is the deployment target: at 2.7 M parameters it has roughly 9× fewer weights than the dataset paper's ResNet50 (25.6 M) while taking the identical pooled input, which makes the accuracy comparison fair.

All variants were trained from scratch on quadrilateral-warped ACPDS patches with the configuration in Table 2; the near-balanced class distribution removed the need for class weighting.

Setting	Value
Epochs	50 (patience 20, early stop)
Batch size	64
Optimizer	AdamW, cosine LR schedule
LR (initial / final)	0.001 / 0.00001
Dropout	0.1
Device	Apple Silicon MPS
Augmentation	RandAugment + h-flip + HSV jitter + random erasing (p=0.2)

Table 2: Stage 2 training configuration.

3.3. Edge inference pipeline

The trained `best.pt` weights are loaded by `edge/detect.py`, which runs on still images, image folders, or a live camera. At startup the script loads the published polygon layout (from `GET /map` when a backend is available), warps every spot, batches the patches through the classifier, and applies temporal smoothing so a single misclassified frame cannot flip a spot's reported state. The `-post` flag pushes the JSON payload to the backend after each pass:

```
{
  "spots":      { "spot_1": "free", "
      spot_2": "occupied" },
  "confidence": { "spot_1": 0.91,   "
      spot_2": 0.84 },
  "timestamp":  "2026-04-21T00:00:00Z"
}
```

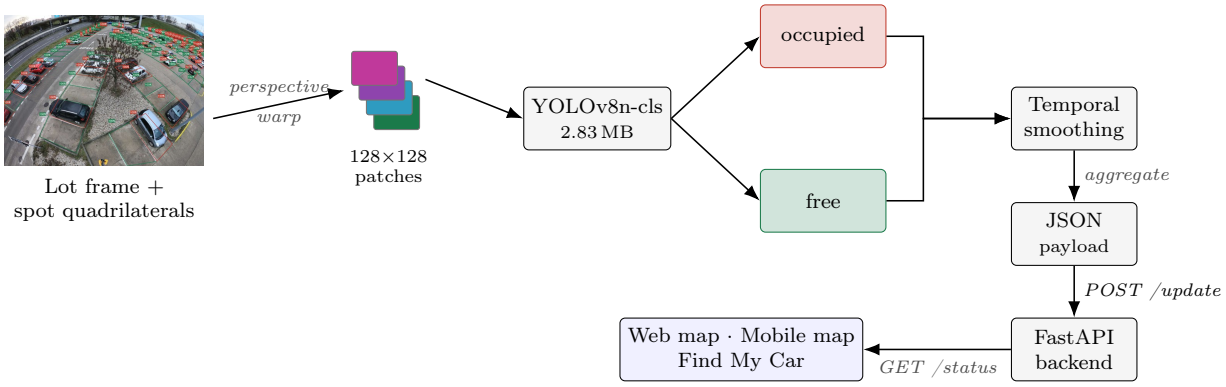
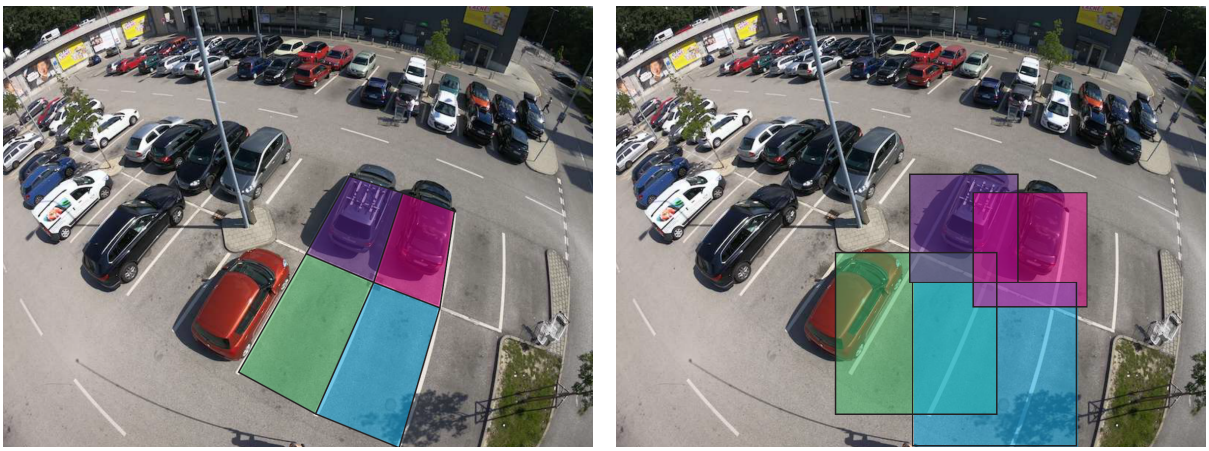


Figure 1: End-to-end system architecture. A lot frame with quadrilateral spot annotations is pooled into perspective-corrected 128×128 patches, classified by YOLOv8n-cls, temporally smoothed, and reported as compact JSON to the FastAPI backend, which drives the web/mobile occupancy maps and Find My Car. Only the JSON leaves the edge device.



(a) "quadrilateral" pooling (b) "square" pooling

Figure 2: Quadrilateral vs. square pooling of parking spaces (reproduced from Marek [3]). The quadrilateral region (a) contains only the target spot's pixels; the bounding square (b) — geometrically equivalent to an axis-aligned ROI — bleeds into neighboring spots and remains perspective-distorted.

Figure 3 shows an annotated output frame, with each quadrilateral colored by predicted state and labeled with its confidence. Because only this JSON crosses the network, the reporting path keeps a measured 99.9 % bandwidth reduction relative to a 1080p H.264 stream, with no camera footage leaving the device.

3.4. Backend and software design

A FastAPI backend with a SQLite store ties the pieces together. The edge device posts occupancy to `POST /update` and clients read it from `GET /status`; the lot layout is published with `POST /map` (alias `/layout`) and served from `GET /map`, with `GET /map/background` returning the bird's-eye-view backdrop. Owner setup uploads photos to `POST /layout`, which runs the Structure-from-Motion pass *server-side* to build the bird's-eye map and extract spot polygons; if it cannot produce a usable layout it returns `422` and the app falls back to manual polygon submission via `POST /map`. A `PATCH /spots/{id}` route lets the owner correct spot labels after generation. The *Find My Car* flow adds `POST /park` (upload a driver photo, run localization, store a session) and `GET /find/{session_id}` (return the matched spot and its corner coordinates), with per-spot reference photos managed through `POST /spots/{id}/references`. Owner-mutating routes can be protected by optional bearer-token authentication (`AUTH_ENABLED`), which also scopes *Find My Car* sessions to their owner. The same published layout drives the edge runtime, the web map, and the mobile map, so all three stay consistent by construction.

Figure 3: Annotated edge inference output (`runs/output/detection.jpg`). Quadrilateral spot polygons are colored by predicted state (green free, red occupied) with per-spot confidence; on this ACPDS lot the promoted classifier reads 70 free / 50 occupied.

4. Result analyses

4.1. Dataset: ACPDS

All training and evaluation use ACPDS, the Action-Camera Parking Dataset [3], captured with a GoPro Hero 6 on a ∼12 m mast — a vantage point close to a real lamppost-mounted camera. It contains 293 high-resolution lot images with 11,236 parking spaces, each annotated as a four-corner quadrilateral rather than a bounding box, which is exactly what makes the perspective-warp pooling of Section 3.1 possible. Crucially, the train / validation / test splits use *disjoint parking lots* (231 / 35 / 27 images; 7,842 / 1,904 / 1,490 spaces), so test accuracy measures generalization to lots never seen in training rather than memorization of a fixed scene — unlike PKLot [1] and CNRPark-EXT, whose same-lot splits inflate reported accuracy. After warping each quadrilateral to a 128×128 patch, the Stage 2 dataset is near-balanced (∼52 % free / 48 % occupied: 3,902 / 3,940 train, 1,073 / 831 val, 885 / 605 test, free / occupied). ACPDS is MIT-licensed.

4.2. Validation accuracy and the retraining gain

Table 3 compares the earlier fixed-ROI baseline with the retrained quadrilateral-warp classifiers on the validation lots, including a controlled square-crop ablation of the nano variant.

Three findings stand out. First, the accuracy jump from the earlier deployment (90.9 %) to the current pipeline (98.3 %) comes overwhelmingly from the input representation, not from model capacity: after the warp normalizes perspective and removes neighboring-spot pixels, the *smallest* variant wins, and the larger m-variant trails it. Second, the square-crop ablation isolates the pooling effect directly: 98.27 % vs. 96.59 % is a 1.7-point gap on identical data and architecture (the square run also converged more slowly and stopped early, so this is a lower bound at this budget). Third, training is stable — Figure 4 shows smooth loss decay with the accuracy plateau reached within roughly 20 epochs, and Figure 6 shows the same for the larger variants — and the confusion matrix in Figure 5 shows balanced class performance.

4.3. Test-split model comparison

Table 4 reports every trained model on the held-out ACPDS test split — 1,490 patches from parking lots in neither the training nor the validation sets, the number that measures genuine generalization — alongside the paper's ResNet50 baselines and the INT8 ONNX

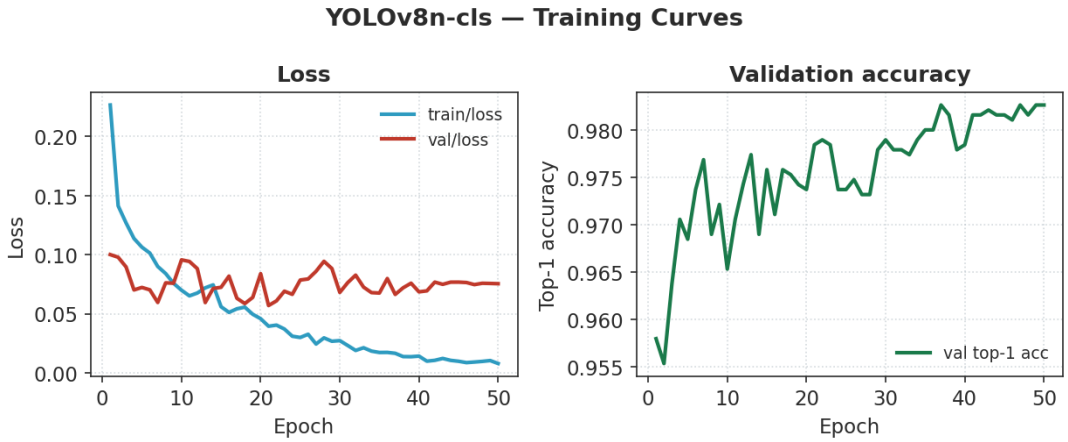
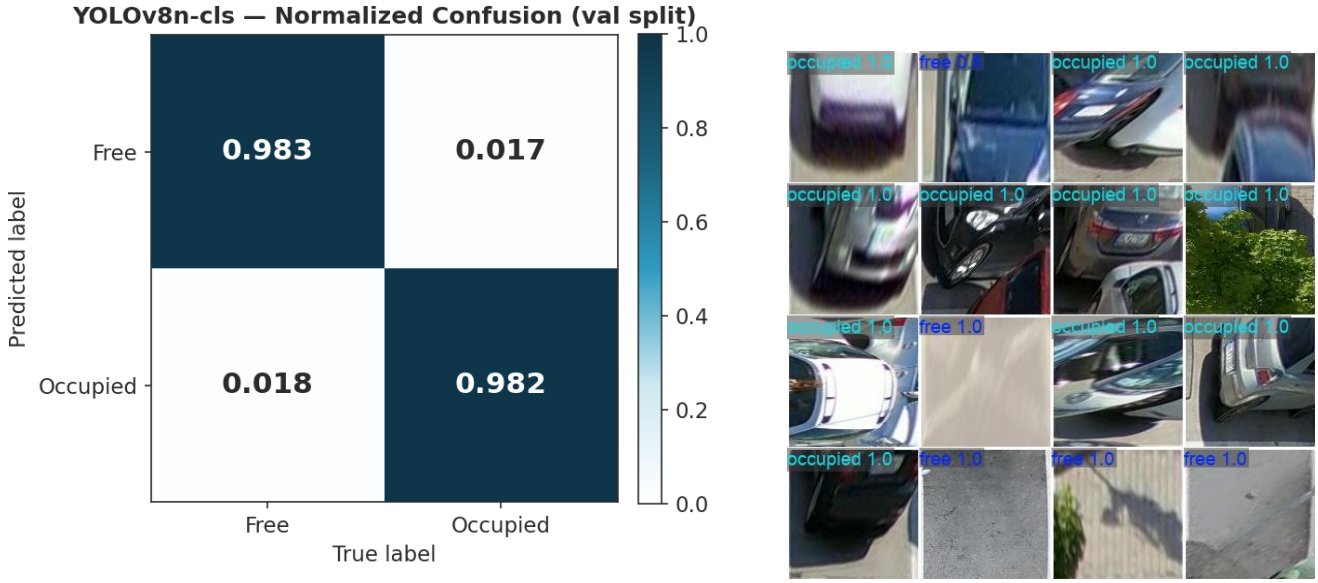


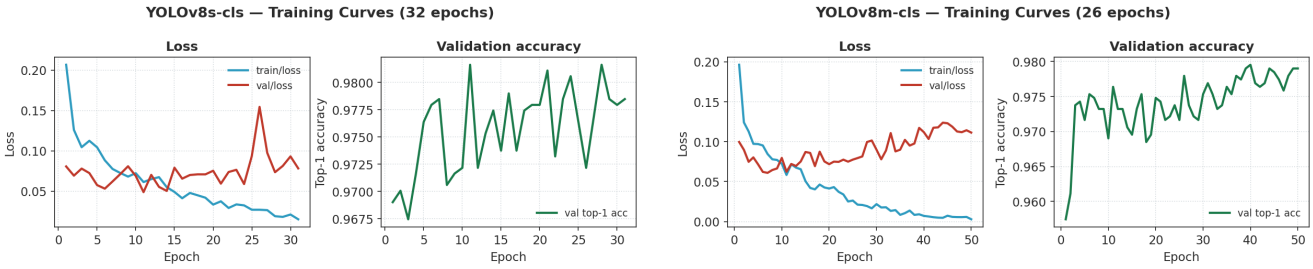
Figure 4: YOLOv8n-cls training curves on warped ACPDS patches (`runs/acpds_cls/yolov8n_stage2`): train/validation loss and top-1 accuracy over 50 epochs.



(a) Normalized confusion matrix (validation split).

(b) Validation-batch predictions.

Figure 5: YOLOv8n-cls qualitative results on the *validation* split. The confusion matrix (a) shows balanced performance across the *occupied* and *free* classes (the held-out *test*-split matrix is reported separately in Figure 8); (b) shows predicted labels on warped validation patches.



(a) YOLOv8s-cls (32 epochs).

(b) YOLOv8m-cls (26 epochs).

Figure 6: Training curves for the comparison variants: train/validation loss and validation top-1 accuracy.

Model	Patch extraction	Epochs	Best top-1 val. acc.
Earlier baseline (rectangular crops, threshold-calibrated)			
YOLOv8m-cls	fixed ROI crops	—	90.85 %[a]
Current (quadrilateral-warp retraining)			
YOLOv8n-cls	quadrilateral warp	50	**98.27 %**
YOLOv8s-cls	quadrilateral warp	31	98.16 %
YOLOv8m-cls	quadrilateral warp	50	97.95 %
YOLOv8n-cls	square crop (ablation)	11	96.59 %
ResNet50 (ACPDS paper [3])	quadrilateral warp	—	∼98 %

Table 3: Stage 2 classification on ACPDS validation lots. [a] The baseline figure is accuracy at the deployed threshold 0.3; it is included for trajectory, not as a strictly like-for-like comparison.

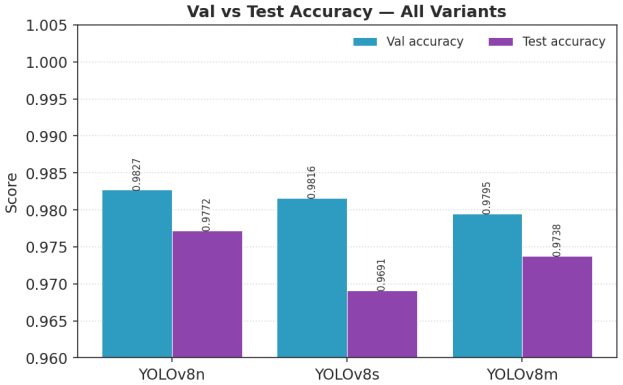


Figure 7: Validation vs. test accuracy across the three YOLOv8 variants. Drops are modest and consistent, indicating distribution shift rather than overfitting.

export. The key finding is that **YOLOv8n-cls outperforms the larger s and m variants on the test split** despite having 2.4–6.3× fewer parameters, and the INT8 export reproduces the PyTorch accuracy exactly — quantization introduces no measurable loss at this model size. The 98 % paper target on the test split is narrowly missed by 0.28 pp; the evidence in the following subsections points to patch quality and distribution shift, not model capacity, as the remaining bottleneck.

4.4. Validation vs. test gap

All three variants show a small, consistent drop from validation to test (Table 5, Figure 7): on average 0.79 pp in accuracy and 3.27 pp in recall. The consistency across model sizes rules out model-specific overfitting and points to distribution shift between the validation and test lots. The recall drop exceeding the accuracy drop matches the deployed error mode — occupied spots are occasionally missed on border-heavy or partial-vehicle patches in the harder test lots — and the larger s/m classifiers do not close the gap.

4.5. Confusion matrix and PR curve

Figure 8 shows the promoted YOLOv8n-cls confusion matrix on the test split. The dominant error mode is false negatives: 26 occupied spaces predicted free (4.3 % of occupied), versus only 8 false positives (0.9 % of free). This is the expected pattern for ACPDS at lamppost height — partially visible vehicles and small bumper/wheel fragments inside the warped quadrilateral — and it shows the model is conservative about declaring a space occupied. Table 6 gives the per-class metrics.

Metric	Free	Occupied	Overall
Precision	0.9712	0.9864	—
Recall	0.9910	0.9570	—
F1	0.9810	0.9715	0.9762 (macro)
Support	885	605	1490

Table 6: Per-class test metrics for the promoted YOLOv8n-cls model.

The precision–recall curve (Figure 9) confirms a high-precision operating region. At the default threshold of 0.5 recall is 0.957; raising the threshold toward 0.7–0.8 would trade recall for a marginal precision gain. For a live system where false *occupied* signals frustrate drivers more than missed free spaces, operating near threshold 0.5 is the right trade-off.

4.6. Per-weather accuracy

The test split was bucketed into sunny, overcast, and low-light thirds by image luminance (HSV V-channel tertiles). Accuracy degrades monotonically from sunny (98.94 %) to low-light (96.63 %), a 2.3 pp drop (Table 7, Figure 10). The effect is asymmetric: under low-light, precision actually *rises* to 99.6 % while recall falls to 93.5 %, i.e. the model becomes very conservative — lower contrast and reduced texture make partially vis-

Model	Pool	Test	Val	Test F1	Params
ResNet50 R-CNN*	sq.	97.97	98.38	—	25.6 M
ResNet50 FPN*	sq.	98.51	98.58	—	25.6 M
YOLOv8n-cls (prom.)	quad	**97.72**	98.27	0.9715	2.7 M
YOLOv8s-cls	quad	96.91	98.16	0.9615	6.4 M
YOLOv8m-cls	quad	97.38	97.95	0.9672	17 M
YOLOv8n INT8 ONNX	quad	97.72	98.27	0.9715	2.7 M
YOLOv8n-cls (abl.)	sq.	96.38	—	0.9559	2.7 M

Table 4: Test-split results on ACPDS (1,490 unseen-lot patches). Accuracies in %. * ResNet50 figures from Table 1 of Marek [3].

Model	Val	Test	Δ	Val R	Test R
YOLOv8n-cls	98.27	97.72	−0.55	98.19	95.70
YOLOv8s-cls	98.16	96.91	−1.25	98.56	94.88
YOLOv8m-cls	97.95	97.38	−0.57	98.68	95.04

Table 5: Validation vs. test accuracy and recall (%). Δ is the accuracy delta; R denotes recall.

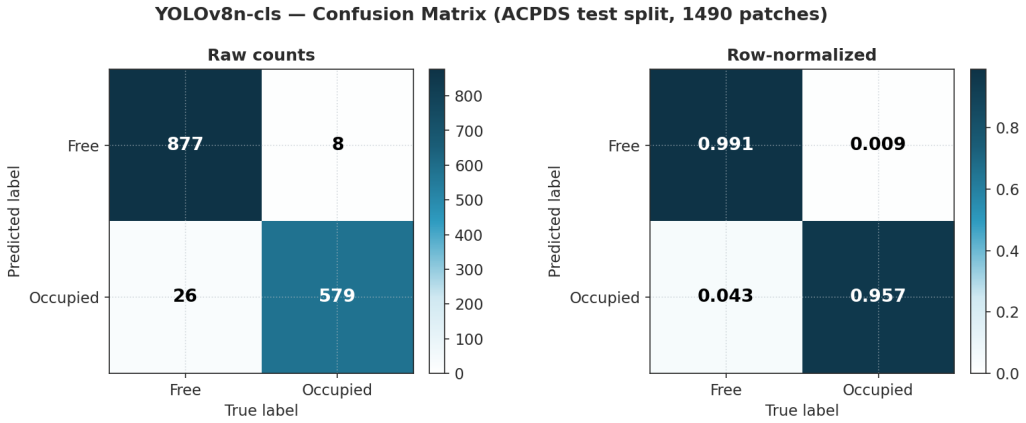


Figure 8: YOLOv8n-cls confusion matrix on the held-out *test* split (distinct from the validation-split matrix in Figure 5). Left: raw counts (TN = 877, FP = 8, FN = 26, TP = 579). Right: row-normalized. Regenerated directly from the promoted checkpoint by `ml/make_report_figures.py` (`make figures`).

ible vehicles harder to distinguish from empty asphalt in the warped patch. Luminance- aware augmentation is a natural way to close this gap.

Condition	N	Acc.	Prec.	Recall
Sunny ($V > 124.5$)	470	98.94	98.32	98.88
Overcast (112.9–124.5)	486	97.74	97.53	95.76
Low-light ($V < 112.9$)	534	96.63	99.59	93.51

Table 7: Per-weather test metrics (%) for YOLOv8n-cls, bucketed by image luminance.

4.7. Pooling comparison on the test split

The validation ablation in Table 3 is corroborated on the held-out test split: the same YOLOv8n-cls architecture trained on quadrilateral- vs. square-pooled patches (Table 8, Figure 11) gives quadrilateral pooling a +1.34 pp accuracy, +4.13 pp precision, and +1.56 pp F1 advantage. Square pooling wins only on recall (+0.99 pp), because the surrounding context in the bounding square occasionally helps detect partially visible vehicles near the spot edges — but that same context injects noise from neighboring spots and degrades precision, the costlier error here.

Pooling	Acc.	Prec.	Recall	F1
Quadrilateral (a)	**97.72**	**98.64**	95.70	**97.15**
Square (b)	96.38	94.51	**96.69**	95.59
Δ (sq. − quad)	−1.34	−4.13	+0.99	−1.56

Table 8: Quadrilateral vs. square pooling on the test split (%), same architecture and data.

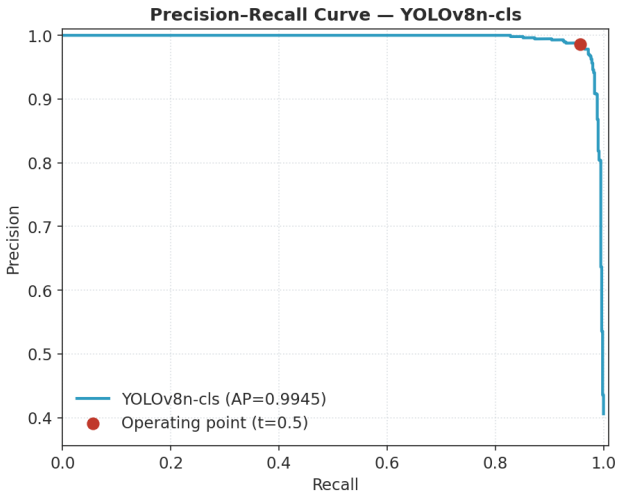


Figure 9: Precision–recall curve for YOLOv8n-cls on the test split (average precision 0.9945); the threshold-0.5 operating point is marked. Regenerated from the promoted checkpoint by `ml/make_report_figures.py` (`make figures`).

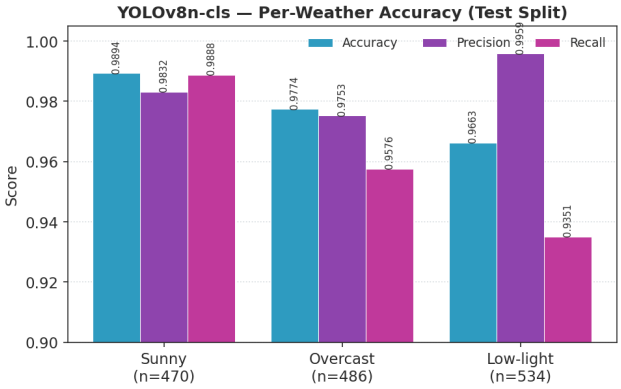


Figure 10: Per-weather accuracy, precision, and recall for YOLOv8n-cls on the test split.

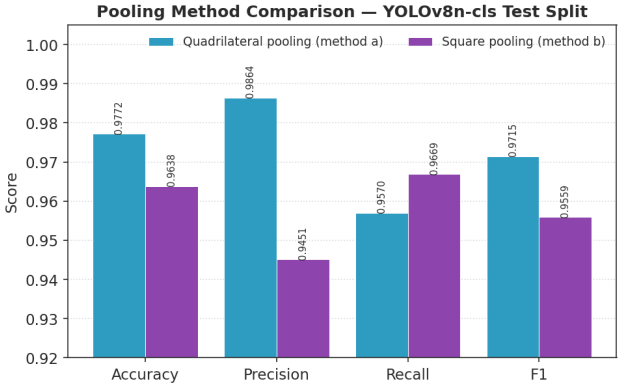


Figure 11: Quadrilateral vs. square pooling on the test split. Quad wins on all metrics except recall.

Notably, this reverses the *direction* of ACPDS Table 2, where square pooling beat quadrilateral pooling for the paper's R-CNN architecture. The reversal is consistent with the architectural difference: R-CNN benefits from surrounding context through its backbone, whereas YOLOv8-cls sees only the single patch, so a cleaner perspective-corrected input is more valuable to it.

4.8. Find My Car localization accuracy

Find My Car matches SIFT features from a driver's photo against per-spot reference images, with FLANN matching and RANSAC homography verification; the spot with the highest inlier count is the top-1 prediction and the inlier count is the confidence. It runs on CPU only and needs no training. SIFT-only is the final scope of this work: it meets the targets without training, so the MobileNetV3 embedding path (documented as an optional future upgrade for day/night robustness) is intentionally out of scope.

Localization was evaluated on a labelled set of 21 night-overhead photos (`query_set.night_overhead.json`), including the three reference timestamps as sanity-check queries. The pipeline scored 21/21 on both top-1 and top-3, comfortably clearing the targets (Table 9). The ∼536 ms average is marginally above the 500 ms target, but the target is conservative for this feature: *Find My Car* runs *once* on arrival and once on return, not in a continuous loop, so 536 ms is a single user-facing wait rather than a throughput constraint.

Metric	Target	Result (21 queries)
Top-1 accuracy	$> 85\,\%$	$100\,\%$ (21/21)
Top-3 accuracy	$> 95\,\%$	$100\,\%$ (21/21)
Avg. runtime	< 500 ms	∼536 ms
GPU required	No	CPU only — met

Table 9: Find My Car localization results on the labelled night-overhead query set (21 queries).

4.9. Inference speed and deployment

Stage 2 inference was benchmarked on the edge device (Apple Silicon MacBook) over 200 timed single-patch inferences after 10 warmup iterations per backend (Table 10). Every backend runs far above the system's needs: the edge runtime posts occupancy at roughly 0.5 Hz, so even the slowest path (PyTorch MPS at 155 FPS) has a ∼300× throughput margin. Two results are worth noting. First, CPU *outpaces* MPS for these 128×128 patches (207 vs. 155 FPS): the patch is so small that GPU dispatch overhead dominates and the device never saturates, so the MPS advantage that shows up

Backend	Latency (ms)	FPS
PyTorch MPS	6.5	155
PyTorch CPU	4.8	207
ONNX FP32 (CPU)	3.3	302
Core ML INT8 (Apple Silicon)*	**1.2**	**858**

Table 10: Measured Stage 2 (`yolov8n-cls`, 128 × 128) inference speed by backend: mean of 200 single-patch runs after 10 warmups. * The ONNX INT8 export could not be benchmarked on the installed ONNX Runtime (the CPU execution provider lacks a `ConvInteger` implementation), so the Core ML INT8 export — the deployed quantized artifact — is reported in its place.

on full-frame models disappears here. Second, the Core ML INT8 export — the actual quantized Apple-Silicon deployment artifact — is fastest at 858 FPS (1.2 ms), with no accuracy loss relative to the PyTorch checkpoint.

5. Discussion

5.1. What worked

Two design choices drove the result. First, the smallest classifier won: `yolov8n-cls` reaches 97.72% test accuracy, ahead of `yolov8s` (96.91%) and `yolov8m` (97.38%) (Table 4), while being roughly 9× smaller in parameters than the ResNet50 paper baseline. Capacity was not the constraint, so the 2.83 MB nano checkpoint is the one we promoted and deployed. Second, quadrilateral perspective-warp pooling beat the simpler bounding-square crop by +1.34 pp test accuracy (97.72% vs. 96.38%; Table 8). Both effects reproduce on the held-out test lots, and the pooling gain has a direct physical explanation: the warp removes perspective distortion and excludes neighbouring vehicles that a loose rectangle would include.

5.2. Limitations

The dominant limitation is *patch quality*, not model capacity. The test-split confusion matrix (Section 4.2, Table 6) shows the residual error is concentrated in 26 occupied→free misses out of 605 occupied patches: these are predominantly partially visible vehicles and thin border fragments introduced by the warp, exactly the cases where larger variants do not help. The validation-to-test drop is small (−0.55 pp accuracy for the nano model) and consistent across `n/s/m` (Table 5), which points to distribution shift across ACPDS's unique-lot splits rather than overfitting. Robustness degrades gracefully with light: accuracy falls monotonically from sunny to low-light (∼2.3 pp; Table 7), where the model becomes conservative — recall drops while precision stays high — rather than failing outright.

Two scope limits remain. The *Find My Car* localization evaluation, while perfect on its set (21/21 top-1; Section 4.8), uses a small, single-condition (night-overhead) query set, so the headline number should not be read as all-weather accuracy. And label ambiguity in borderline patches caps the achievable ceiling for any classifier on this data.

5.3. Production considerations

The deployed path has ample operating margin. The Core ML INT8 export runs at 858 FPS (1.2 ms per patch; Table 10), about 300× the ∼0.5 Hz reporting cadence, and the edge node sends only compact occupancy JSON, a ∼99.9% bandwidth reduction versus H.264 video (Section 3.4). Temporal smoothing absorbs per-frame flicker, and a 30-minute soak test on the live quadrilateral path completed without crashes. On the product side, *Find My Car* ships SIFT-only: it is training-free, CPU-only, and sufficient for the evaluated set; MobileNetV3 embeddings remain a documented optional upgrade for broader lighting robustness rather than a requirement of this work.

6. Application description

The product is delivered as a web app (React + Leaflet) and a mobile app (React Native / Expo) over the FastAPI backend of Section 3.4. The platform split is deliberate: owner setup is web-only, *Find My Car* is mobile-only, and the live occupancy map ships on both.

6.1. Owner setup (web)

The owner uploads photos of the lot to `POST /layout`; the backend runs the SfM pass server-side to compute a bird's-eye-view layout and extract quadrilateral spot polygons, persists it, and returns it for preview. If SfM cannot build a usable layout the route returns `422` and the app falls back to manual polygon submission. The owner can then correct each spot's label inline (persisted via `PATCH /spots/{id}`) before the layout drives every other screen. This is the one-time configuration step that every other screen depends on. The screenshot in Figure 12 shows the web flow driving a sample placeholder layout (`spot_source: placeholder_grid`); the quadrilateral extraction itself is demonstrated on real ACPDS imagery in Figures 1 and 3. The current SfM stage uses ORB feature matching with homography blending; a higher-fidelity bird's-eye reconstruction remains future work (Section 7).

6.2. Live occupancy map (web and mobile)

Both clients poll `GET /status` and color each published spot by its current state, with a free / occupied /

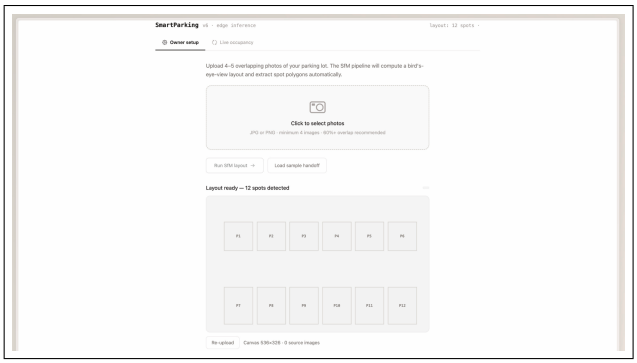


Figure 12: Owner setup (web): the photo-upload area and the resulting 12-spot layout panel (*"Layout ready — 12 spots detected"*), ready to publish as the lot map. The spots shown are a sample placeholder grid (`spot_source: placeholder_grid`) used to exercise the UI; real quadrilateral extraction on ACPDS imagery is shown in Figures 1 and 3.

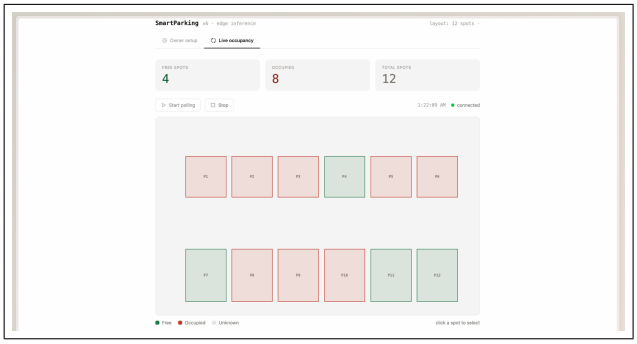


Figure 13: Web live occupancy map: each published spot colored green (free) or red (occupied), with free / occupied / total counters, a polling control, and a live connection indicator.

total summary. The mobile map renders the same layout as coordinate-accurate SVG polygons with pinch-to-zoom and pan. Counts are scoped to the active layout's spot IDs (see Section 6.4); in Figure 13 the 4 / 8 / 12 summary agrees with the colored spots.

6.3. Find My Car (mobile)

On arrival the driver taps *Park Here*; the app uploads a photo to `POST /park`, which localizes it against the per-spot reference photos (managed through `POST /spots/{id}/references`, with the bundled sample set as fallback) and returns a `session_id`. Later the driver taps *Find My Car*; the app calls `GET /find/{session_id}` and highlights the matched spot in amber on the map.

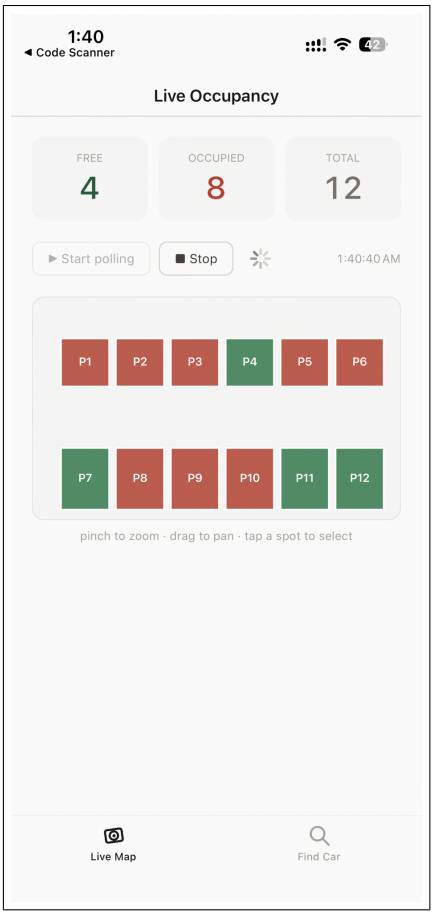


Figure 14: Mobile live occupancy map (SVG polygons with pinch-to-zoom and pan). The 4 / 8 / 12 free / occupied / total summary and per-spot colors mirror the web client of Figure 13 against the same 12-spot layout.

6.4. Case study: live occupancy counter bug

During integration testing, the web dashboard displayed 55 free and 127 occupied spots against a 12-spot layout. The root cause was that the counters aggregated *every* entry of the raw `/status` payload — which can contain stale spot identifiers from earlier edge runs — while the map rendered only the published layout's spots. The fix constrains the counters to the layout's spot identifiers, restoring the invariant free + occupied $\leq$ total and matching the mobile implementation, which was already correct. The episode motivated a hardening recommendation: the backend should filter `/status` responses to the currently published layout so all clients receive consistent data by construction.

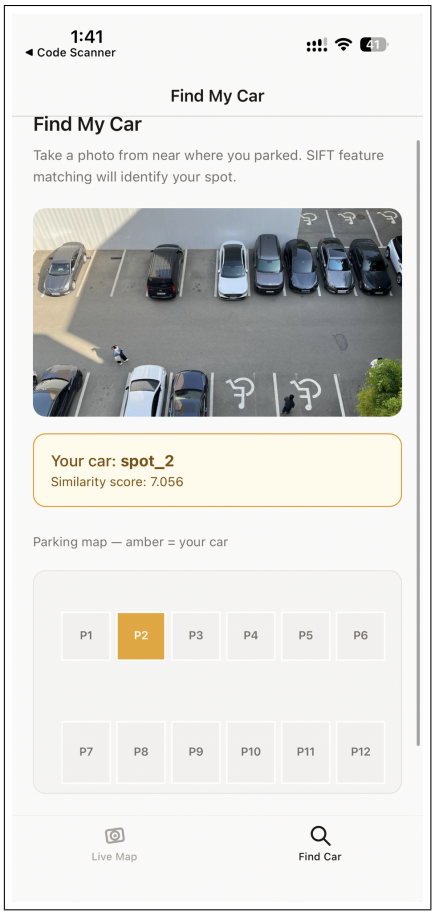


Figure 15: Find My Car (mobile): the driver's overhead photo is matched by SIFT feature matching to the published layout (here `spot_2`, similarity score 7.056), and the localized spot is highlighted in amber on the map.

7. Conclusion

This project delivers an edge-based smart parking system that classifies per-space occupancy from a single overhead camera and reports it as a few hundred bytes of JSON, with no imagery leaving the device. The decisive design choice was to pool each parking space along its annotated quadrilateral and perspective-warp it before classification: that single change lifted accuracy from a 90.9 % fixed-ROI baseline to 97.72 % on ACPDS's held-out, unseen-lot test split — matching the dataset paper's ResNet50 baseline with the *smallest* model variant — and a controlled ablation confirms the quadrilateral advantage holds on both validation and test. Around the model, the system is a complete product: owner setup, live web and mobile occupancy maps, and a photo-based *Find My Car* feature that localizes 21/21 queries correctly. Stage 2 runs at 155–858 FPS across PyTorch, ONNX, and Core ML INT8 backends, hundreds of times faster than the reporting cadence requires.

The full product path is validated automatically: an in-process smoke test drives owner setup, map persistence, edge update, live status, the park/find round-trip against the real localizer, owner label correction, and per-spot reference management, and the clients carry contract tests for the layout, occupancy-counting, label-correction, and park/find response shapes.

Future work is narrow. Low-light recall is the weakest metric (93.5 %), and luminance-aware augmentation is the natural remedy. Owner setup now runs Structure-from-Motion on the uploaded photos server-side with a manual-correction fallback; the remaining gap is reconstruction fidelity, as the current ORB-based bird's-eye stage is coarse and a metric SfM or learned homography would sharpen it. A longer soak test and backend-side filtering of status payloads against the published layout (the hardening from Section 6.4) round out the remaining engineering work.

References

[1] Paulo R. L. Almeida, Luiz S. Oliveira, Alceu S. Britto Jr., Eunelson J. Silva Jr., and Alessandro L. Koerich. PKLot – a robust dataset for parking lot classification. *Expert Systems with Applications*, 42(11):4937–4949, 2015. 4

[2] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. `https://docs.ultralytics.com`, 2023. 1

[3] Martin Marek. Image-based parking space occupancy classification: Dataset and baseline, 2021. arXiv:2107.12207. 2, 3, 4, 6, 7